

System Architecture

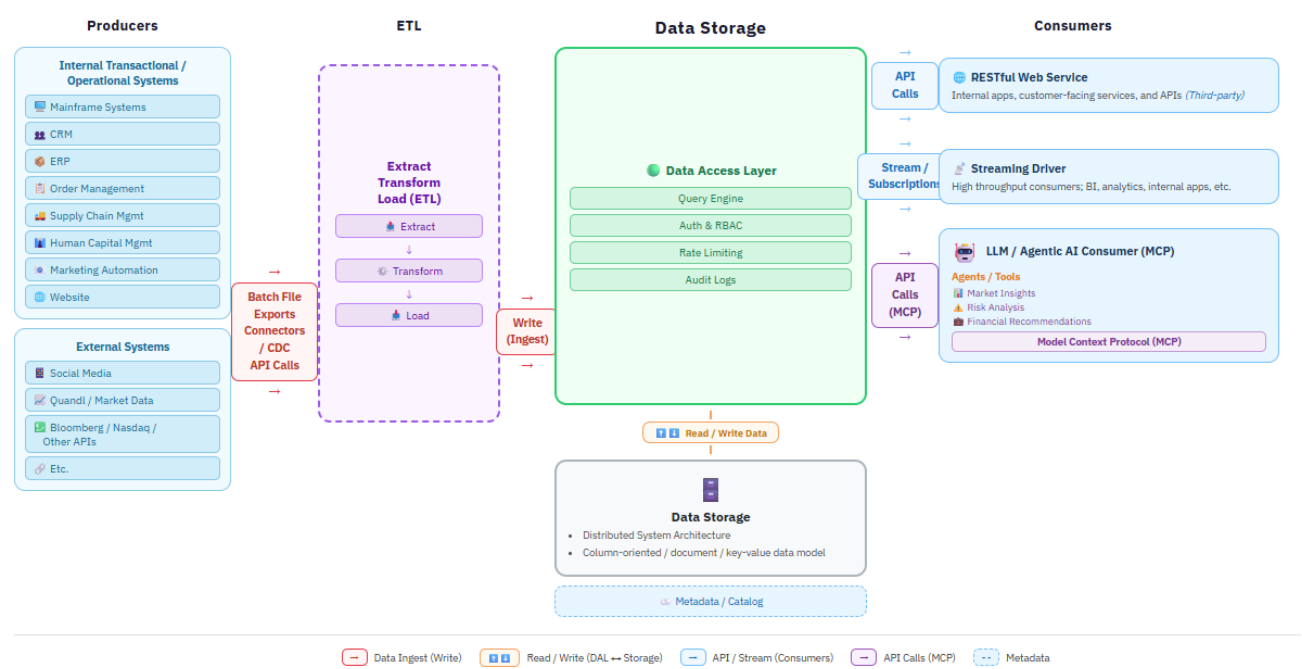
System Architecture1

System Architecture1

Java Tech Stack Overview.....3

Overall System Architecture

The diagram below illustrates the overall system architecture



Producers

The are the external sources of data for a data warehouse platform. These may be either *Internal Systems*, such as transactional / operational systems, or core business systems (CRM, ERP, etc.) that generate transactional data as part of day-to-day operations, or *External Systems*, like 3rd party data sources (e.g., financial data providers/Nasdaq Data Link), social media platforms, or other APIs that feed external data into the platform.

ETL (Extract / Transform / Load)

This is the pipeline that ingests raw data from producers, applies transformations and cleansing rules, then loads the processed data into the data warehouse platform for storage and consumption.

Data Access Layer (DAL)

The unified gateway through which all consumers access data. Enforces query execution, authentication, role-based access control (RBAC), rate limiting, and audit logging — ensuring secure and governed data access. It allows clients (other modules of the application, e.g. a RESTful Web Service, ETL) to ingest/retrieve the data to/from the data warehouse. Its main responsibility is to hide the underlying details (e.g., the database management system in-use

(Cassandra/Mongo/other), the object-database mapping library etc.). It should expose a CRUD-like interface enabling clients to easily add/retrieve time series data from the data warehouse.

Data Storage

The physical persistence layer built on a distributed data storage system (e.g., Apache Cassandra, Mongo DB, Microsoft Cosmos DB etc.) supporting column-oriented / document / key-value data models to handle diverse workloads at scale. Metadata / System Catalogue is an intrinsic component of the DBMS that stores schema definitions, data lineage, and discovery metadata, enabling consumers (such as DAL) to understand and locate available data.

Consumers

A data warehouse platform makes available its data to external consumers that enables Business Intelligence (BI) at scale, the process of big data-enabled decision making in organisations.

- RESTful Web API — Exposes data via standard RESTful APIs for internal applications, customer-facing services, and third-party integrations.
- Streaming Driver — Provides high-throughput, real-time data streams via subscriptions, serving BI tools, analytics engines, and internal applications.
- LLM / Agentic AI Consumer (MCP) — AI agents powered by large language models that consume data via the Model Context Protocol (MCP) to perform intelligent tasks such as market insights, risk analysis, or financial recommendations.

Scalable Architecture for Big Data

A scalable architecture for large datasets processing typically includes¹:

- Pagination for API queries – this translates downstream into pageable queries against the database
- Streaming for large data processing – from REST API contract down to database records fetching use streams for minimal memory usage and continuous processing
- Batch operations for database writes – reduce database roundtrips, perfect for ETL processes
- Indexed database tables – useful for search, sort and filter when reading large record sets; balance index costs against benefits
- Asynchronous job processing – asynchronous processing, message queues (RabbitMQ) or streaming platforms (Apache Kafka) decouple producers from consumers, improve fault tolerance and enable workloads distribution across multiple services

Conversely, when working with large datasets, avoid these common pitfalls:

- Loading entire tables into memory
- Executing thousands of individual database queries

¹ This Medium post (<https://medium.com/@csvaibhavpasrija/d546e4329ade>) describes into more details the design guidelines proposed in this section

- Ignoring database indexing
- Using blocking synchronous workflows
- Returning massive datasets through APIs

The proposed architecture in this section follows these guidelines and adapts them to NoSQL data storage systems.

Java Tech Stack Overview

The table below consolidates the technologies across all layers of the platform. Alternatives are provided for the NoSQL database (Apache Cassandra 5.0 / MongoDB) and the build tool (Gradle 8.x / Apache Maven). Each entry includes a reference to the official documentation.

Layer	Technology	Reference
Application Framework	Spring Boot 4.0.x	https://spring.io/projects/spring-boot
NoSQL Database (Option 1)	MongoDB 7.x	https://www.mongodb.com/docs/drivers/java/
NoSQL Database (Option 2)	Apache Cassandra 5.0	https://cassandra.apache.org/doc/latest/
Data Access (MongoDB)	Spring Data MongoDB 5.x	https://spring.io/projects/spring-data-mongodb
Data Access (Cassandra)	Spring Data Cassandra 5.x	https://spring.io/projects/spring-data-cassandra
REST API Docs	SpringDoc OpenAPI 2.x	https://springdoc.org
DTO Mapping	MapStruct 1.7.x	https://mapstruct.org
ETL Streaming Platform (Ingestion)	Apache Kafka 4.x	https://kafka.apache.org/
Big Data Analytics	Apache Spark 4.1.x + MongoDB Connector / Cassandra Connector	https://spark.apache.org/docs/latest
LLM Abstraction	Spring AI 1.1.x	https://spring.io/projects/spring-ai
MCP Server	MCP Java SDK	https://github.com/modelcontextprotocol/java-sdk
Agentic AI (Bonus)	LangChain4j 0.32.x	https://github.com/langchain4j/langchain4j
Build Tool (Option 1)	Apache Maven 4.x	https://maven.apache.org/index.html

Build Tool (Option 2)	Gradle 9.x	https://docs.gradle.org/current/userguide/userguide.html
Unit Testing	JUnit 5 + Mockito 5.x	https://junit.org/junit5/
Integration Testing	Testcontainers 1.19.x	https://testcontainers.com
API Testing	REST Assured 5.x	https://rest-assured.io
Logging	SLF4J + Logback 2.x	https://logback.qos.ch
Containerization	Docker + Docker Compose	https://docs.docker.com/compose/

NB: This is not a prescriptive bill of material, serves as informative guidelines.